

Homework Set 3

Now with more Linear Algebra

Revision: 26-Feb-2025

3.1 The Steady-State Heat Equation

For an isotropic material at steady-state heat conducting through a body can be modeled as the simple heat equation

$$\nabla^2 T(\mathbf{x}) = 0, \quad \mathbf{x} \in \Omega. \quad (3.1)$$

The boundary conditions can come in many forms, but let's consider the Dirichlet (essential) boundary conditions

$$T = f(\mathbf{x}), \quad \mathbf{x} \in \partial\Omega \quad (3.2)$$

for a known function f , where T is the temperature, and $\mathbf{x}$ are the coordinates of the body Ω . The boundary of Ω is denoted $\partial\Omega$.

In 2D Cartesian coordinates, we can write the equation as

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = 0. \quad (3.3)$$

If we discretize this partial differential equation using 2nd-order central finite-differences, we get the approximation

$$\frac{T_{i+1,j} - 2T_{i,j} + T_{i-1,j}}{\Delta x^2} + \frac{T_{i,j+1} - 2T_{i,j} + T_{i,j-1}}{\Delta y^2} = 0. \quad (3.4)$$

It is important to note that all the temperatures are linearly related to each other. When the grid is square, $\Delta x = \Delta y$, the equation becomes even nicer to gaze on

$$T_{i+1,j} + T_{i-1,j} + T_{i,j+1} + T_{i,j-1} - 4T_{i,j} = 0. \quad (3.5)$$

Now let's use this scheme to solve some heat problems.

3.1.1 A coarse grid

Consider the plate shown in Figure 3.1 with boundary conditions specified on each edge. Using (3.5), along with the boundary conditions on the figure the following 9 equations can be constructed for the 9 unknown temperatures. The units here are all consistent in degrees Celsius.

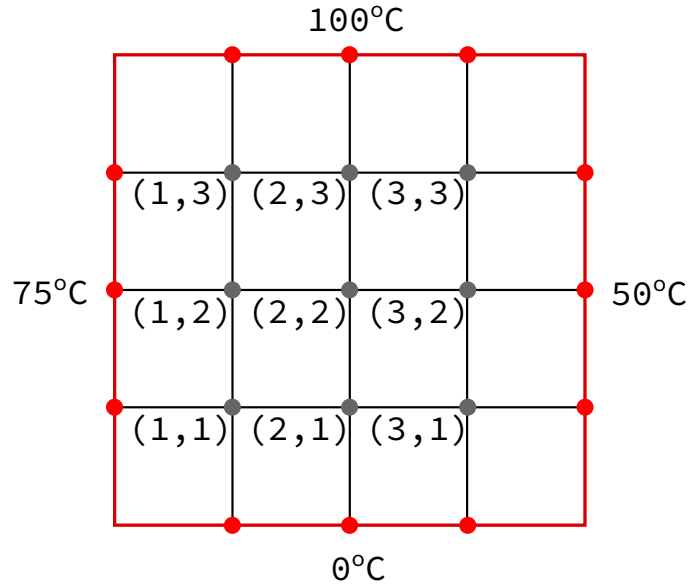


Figure 3.1: A heated plate where the boundary temperatures are held constant and are uniform across each edge.

$$\begin{array}{ccccccccc}
 +4T_{11} & -T_{21} & & -T_{12} & & & & & = & 75 \\
 -T_{11} & +4T_{21} & -T_{31} & & -T_{22} & & & & = & 0 \\
 & -T_{21} & +4T_{31} & & & -T_{32} & & & = & 50 \\
 -T_{11} & & & +4T_{12} & -T_{22} & & -T_{13} & & = & 75 \\
 & -T_{21} & & -T_{12} & +4T_{22} & -T_{32} & & -T_{23} & = & 0 \\
 & & -T_{31} & & -T_{22} & +4T_{32} & & -T_{33} & = & 50 \\
 & & & -T_{12} & & & +4T_{13} & -T_{23} & = & 175 \\
 & & & & -T_{22} & & -T_{13} & +4T_{23} & -T_{33} & = & 100 \\
 & & & & & -T_{32} & & -T_{23} & +4T_{33} & = & 150
 \end{array} \tag{3.6}$$

1. Solve (3.6) using a built-in solver. Such as Matlab's or Julia's \ operator. Plot the results on a contour plot.
2. Determine the condition number (Moler, 2017; Wikipedia contributors, 2018) based on the ratio of the singular values σ of the system matrix you solved in the previous part of the problem.

$$\kappa = \frac{\sigma_{\max}(\mathbf{A})}{\sigma_{\min}(\mathbf{A})} \tag{3.7}$$

Compare κ to the results of a built-in condition number function.

3. Implement the unpreconditioned biconjugate gradient stabilized method, usually written as BiCGSTAB, of van der Vorst (1992). Solve (3.6) using your BiCGSTAB function. During the iterations, keep track of the residuals, and make a plot showing how your solver converged.

3.1.2 Oh that looks fine

Now that we have the easy (small) case under our collective belts, let's consider solving the problem for reals. This means we need to refine the mesh until our solutions do not significantly change. This is referred

to as a *grid convergence study*, and is essentially finding a solution that is as close as we can get to being free from truncation error. Procedure: double the number of points in each direction until the difference between solutions is below a set tolerance. Let's keep track of the center location of the domain as the key point, and see how that point converges to a specific value.

1. Implement a Successive Over Relaxation (SOR) solver for this system of n equally-spaced grid points in each direction.
2. Plot the convergence as a function of grid points. Also make a contour-type plot of the finest grid.
3. Implement the BiCGSTAB method without building the entire matrix $\mathbf{A}$ as a single dense array. Hint: think carefully about what operations are needed in the method and how to do them in terms of (3.5). Compare the following things between SOR and BiCGSTAB:
 - (a) Number of iterations vs grid size
 - (b) Wall time vs grid size
 - (c) For a large grid case, compare how the residuals converge.

3.2 Linear least-squares curve fitting

Consider the problem of fitting a polynomial to some collected data. You have a pile of x 's (independent variable) and associated y 's (dependent variable), and you would like to know the "best" set of coefficients $\{a\}$ in the expression

$$y = a_0 + a_1x + a_2x^2 + \cdots + a_nx^n \quad (3.8)$$

that fits your data. This is the classical linear least-squared error curve-fitting problem. As long as the $\{a\}$ values are linear, then finding them is linear. This means we can generalize this problem to much more complicated functions like

$$y = a_0 + a_1f_1(x) + a_2f_2(x) + \cdots + a_nf_n(x) \quad (3.9)$$

as long as f_i and f_j are linearly independent.

Let's get back to the polynomial, and write out several of the equations we have using the data points $(x_1, y_1), (x_2, y_2), \dots, (x_p, y_p)$

$$\begin{aligned} y_1 &= a_0 + a_1x_1 + a_2x_1^2 + \cdots + a_nx_1^n \\ y_2 &= a_0 + a_1x_2 + a_2x_2^2 + \cdots + a_nx_2^n \\ &\vdots \\ y_p &= a_0 + a_1x_p + a_2x_p^2 + \cdots + a_nx_p^n \end{aligned}$$

We can arrange that into matrix notation

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_p \end{bmatrix} = \begin{bmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^n \\ 1 & x_2 & x_2^2 & \cdots & x_2^n \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_p & x_p^2 & \cdots & x_p^n \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_n \end{bmatrix}$$

We can also write this as

$$\mathbf{y} = \mathbf{X}\mathbf{a} \quad (3.10)$$

where the rectangular matrix $\mathbf{X} \in \mathbb{R}^{p \times n}$ has all the known data, and the unknown parameters to be fit are in $\mathbf{a} \in \mathbb{R}^n$. In order for this to make sense, we intuitively know that we need to have more data points than parameters, so this means that $p > n$ (most problems have $p \gg n$). If we compute the square of the residuals $\mathbf{y} - \mathbf{Xa}$

$$\begin{aligned} J &= (\mathbf{y} - \mathbf{Xa})^T (\mathbf{y} - \mathbf{Xa}) \\ &= \mathbf{y}^T \mathbf{y} - 2\mathbf{a}^T \mathbf{X}^T \mathbf{y} + \mathbf{a}^T \mathbf{X}^T \mathbf{Xa} \end{aligned} \quad (3.11)$$

we want to minimize this error, so we need to find $\mathbf{a}$ such that $\frac{\partial J}{\partial \mathbf{a}} = \mathbf{0}$. Differentiating we get

$$\begin{aligned} \frac{\partial J}{\partial \mathbf{a}} &= \frac{\partial}{\partial \mathbf{a}} (\mathbf{y}^T \mathbf{y} - 2\mathbf{a}^T \mathbf{X}^T \mathbf{y} + \mathbf{a}^T \mathbf{X}^T \mathbf{Xa}) \\ &= \mathbf{0} - 2\mathbf{X}^T \mathbf{y} + \mathbf{X}^T \mathbf{Xa} + (\mathbf{a}^T \mathbf{X}^T \mathbf{X})^T \\ &= 2(-\mathbf{X}^T \mathbf{y} + \mathbf{X}^T \mathbf{Xa}) \end{aligned}$$

Setting this equal to zero gives the necessary condition

$$-\mathbf{X}^T (\mathbf{y} - \mathbf{Xa}) = \mathbf{0} \quad (3.12)$$

1. Show that solving for $\mathbf{a}$ in (3.12) gives the so-called Moore-Penrose inverse (sometimes called the pseudoinverse).

$$\mathbf{a} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} = \mathbf{X}^+ \mathbf{y} \quad (3.13)$$

2. Show that if you compute the singular value decomposition of $\mathbf{X} = \mathbf{USV}^T$, then

$$\mathbf{X}^+ = \mathbf{VS}^+ \mathbf{U}^T \quad (3.14)$$

where $\mathbf{S}^+ = (\mathbf{S}^T \mathbf{S})^{-1} \mathbf{S}^T$ is the pseudoinverse of the diagonal (but rectangular) matrix of singular values. To compute $\mathbf{S}^+$, take the inverse of each non-zero element on the diagonal of $\mathbf{S}$, leaving the zeros in place, and then transpose the matrix.

3. Build a function that outputs $\mathbf{X}$, where the inputs are $\mathbf{x} = \{x_1, x_2, \dots, x_p\}$, the handle of the (general) function to be curvefit f , and the number n . If you're using Matlab, this function definition must be

```
function X = buildX( x, f, n)
```

The interface of the model function to be fit must be

```
function y = f( x, a)
```

If you're using something else, do the equivalent of those function interfaces. The singular value decomposition function is in the `LinearAlgebra`

4. Build a function that takes input $\mathbf{X}$ and $\mathbf{y} = \{y_1, y_2, \dots, y_p\}$, and computes $\mathbf{a}$ from (3.14).

```
function a = mycurvefit( X, y)
```

5. To test out your function, make up several datasets with noise and try to find the true parameters from the data. Try several polynomials of increasing order.

- Linear Example: use the function $y = 2x - 1$ for a range of values of x generate a set of y . Then add noise to this using Matlab's `awgn`¹ (or something equivalent in the language you are using), and run your fitting routine against this noisy dataset. As you increase the number of points p and decrease the signal-to-noise ratio, does your function still return the correct **a**?
6. One of the classic bits of literature in the study of rubber materials is the work of Treloar (1944). Figure 3.2 shows the uniaxial stretching of a particular rubber, in particular let's focus on the dots of curve (a). This curve shows the results of a tensile test of a piece of rubber. The horizontal axis is the stretch ratio $\lambda = l/l_0$, where l is the deformed length and l_0 is the undeformed length. The vertical axis shows the stress in a rather strange unit, but we could take them to be (10^5) Pa.² Use a digitizer to extract numerical values from the marker points (λ_i, σ_i) . I used an open source one called Engauge Digitizer.³

There are many models that try to describe the behavior of rubber such as the Ogden, Mooney-Rivlin, neo-Hookean, and polynomial models. Let's assume we can model this curve with a 5-term polynomial model. After a bit of theory (which I'm skipping here)⁴, the engineering stress of a simple stretch can be written as

$$\sigma_e = 2(\lambda - \lambda^{-2}) \left[C_{10} + C_{01}\lambda^{-1} + 2C_{20}(\lambda^2 + 2\lambda^{-1} - 3) + 2C_{02}(2\lambda + \lambda^{-2} - 3) + 3C_{11}(\lambda - 1 - \lambda^{-1} + \lambda^{-2}) \right] \quad (3.15)$$

where λ is the stretch ratio (change in length over initial length), and C_{ij} are the five constants to be found. Using the data you've extracted, compute values for C_{ij} and comment on the utility of this particular model for this dataset.

¹<https://www.mathworks.com/help/comm/ref/awgn.html>

²For example we would interpret 30 on the graph as 3 MPa.

³Find Engauge here <https://markumitchell.github.io/engauge-digitizer/>

⁴There are many interesting texts on this. For a brief summary of this model see https://www.researchgate.net/publication/301200150_Curve_Fitting_for_Ogden_Yeoh_and_Polynomial_Models

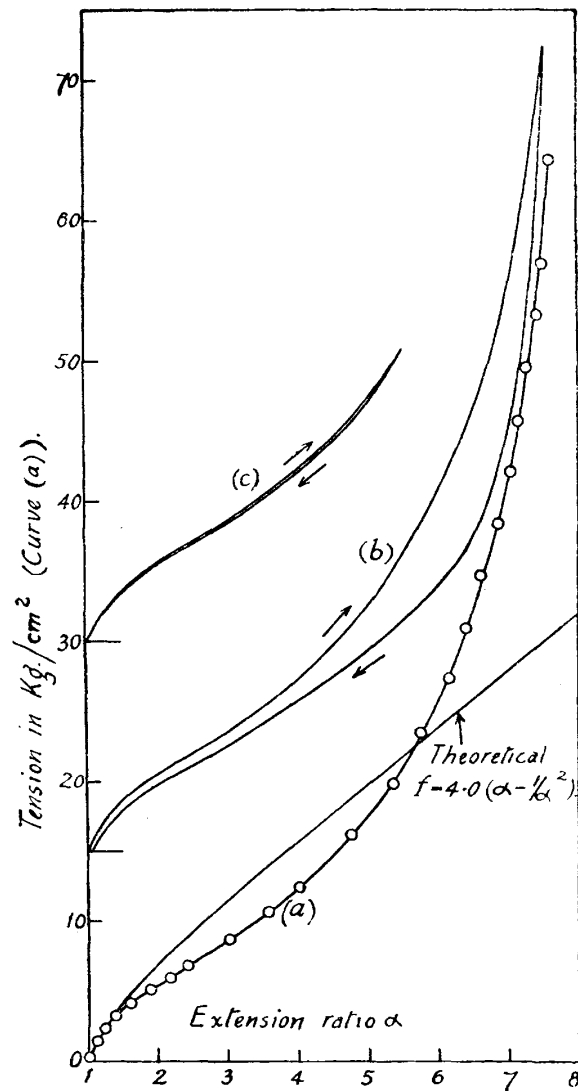


FIG. 3.—Simple elongation. 8 % S rubber, 20° C.

Figure 3.2: A classic dataset on stretching rubber, note that the curve is highly nonlinear and does not look like Hooke's Law.

Bibliography

- Moler, C. (2017). *What is the Condition Number of a Matrix?* MathWorks. URL: <https://blogs.mathworks.com/cleve/2017/07/17/what-is-the-condition-number-of-a-matrix/>.
- Treloar, L. R. G. (1944). “Stress-strain data for vulcanised rubber under various types of deformation”. *Transactions of the Faraday Society*, **40**: pp. 59–70. DOI: 10.1039/tf9444000059.
- van der Vorst, H. A. (1992). “Bi-CGSTAB: A Fast and Smoothly Converging Variant of Bi-CG for the Solution of Nonsymmetric Linear Systems”. *SIAM Journal on Scientific and Statistical Computing*, **13**:2, pp. 631–644. DOI: 10.1137/0913035.
- Wikipedia contributors (2018). *Condition Number*. URL: https://en.wikipedia.org/wiki/Condition_number#Matrices (visited on 01/25/2018).